

Übungsblatt 3

Frontend-Technologien

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

Modul A — HTML

Übung 3.A.1: Semantisches HTML

Gegeben ist folgendes unsemantisches HTML:

```
<div class="header">
  <div class="nav">
    <div class="nav-item"><a href="/">Home</a></div>
    <div class="nav-item"><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="main">
  <div class="article">
    <div class="title">Produktname</div>
    <div class="text">Beschreibung...</div>
  </div>
</div>
<div class="footer">© 2026</div>
```

1. Schreiben Sie die Struktur mit semantischen HTML5-Elementen um.
2. Warum ist semantisches HTML für Suchmaschinen und Screenreader wichtig?

Übung 3.A.1: Semantisches HTML — Lösung

```
<header>
  <nav>
    <a href="/">Home</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h1>Produktname</h1>
    <p>Beschreibung...</p>
  </article>
</main>
<footer>© 2026</footer>
```

Warum semantisch?

- **Suchmaschinen:** Crawler verstehen die Bedeutung der Inhalte — `<h1>` signalisiert Hauptüberschrift, `<nav>` Navigationsbereiche. Das verbessert SEO.
- **Screenreader:** Blinde Nutzer navigieren per Tastatur durch Landmarks (header, main, nav). Ohne Semantik ist jedes `<div>` gleichwertig — Navigation wird unmöglich.

Übung 3.A.2: Formulare und Accessibility

Erstellen Sie ein barrierefreies HTML-Formular für eine Produktsuche mit:

- Texteingabe für den Suchbegriff (Pflichtfeld)
- Dropdown für Kategorie (Elektronik, Kleidung, Bücher)
- Checkbox „Nur verfügbare Produkte“
- Submit-Button

Welche drei HTML-Attribute sind für Barrierefreiheit unverzichtbar?

Übung 3.A.2: Formulare und Accessibility — Lösung

```
<form action="/search" method="GET">
  <label for="query">Suchbegriff</label>
  <input id="query" name="query" type="text" required
    placeholder="z.B. Laptop" aria-required="true"/>

  <label for="category">Kategorie</label>
  <select id="category" name="category">
    <option value="">Alle</option>
    <option value="electronics">Elektronik</option>
    <option value="clothing">Kleidung</option>
    <option value="books">Bücher</option>
  </select>

  <label>
    <input type="checkbox" name="available" value="true"/>
    Nur verfügbare Produkte
  </label>

  <button type="submit">Suchen</button>
</form>
```

Unverzichtbare Attribute:

1. **for / id-Verknüpfung:** Verbindet Label mit Input — Klick auf Label fokussiert das Feld.
2. **required:** Zeigt Pflichtfelder an und verhindert leere Submits (Browser-Validierung).
3. **type (auf <input>):** Bestimmt Eingabemodus und Tastaturlayout auf Mobilgeräten.

Modul B — CSS

Übung 3.B.1: Box-Modell und Selektoren

Gegeben folgendes CSS:

```
.card {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 16px;  
}
```

1. Wie breit ist `.card` im Standard-Box-Modell (content-box)? Begründen Sie.
2. Wie breit wäre `.card` mit `box-sizing: border-box`?
3. Schreiben Sie einen CSS-Selektor, der nur das erste `.card`-Element innerhalb eines `<section>` trifft.

Übung 3.B.1: Box-Modell und Selektoren — Lösung

1. **Standard** (content-box): `width` gilt nur für den Content-Bereich. Gesamtbreite = $300 + 20 + 20 + 2 + 2 = 344 \text{ px}$. Margin gehört nicht zur Elementbreite, aber zum benötigten Platz im Layout.
2. **border-box**: `width: 300px` ist das Gesamtmaß inklusive Padding und Border. Content-Breite = $300 - 40 - 4 = 256 \text{ px}$. Die Gesamtbreite des Elements bleibt **300 px**.

3.

```
section .card:first-child { }  
/* oder präziser: */  
section > .card:first-of-type { }
```

Übung 3.B.2: Responsive Layout mit Flexbox

Implementieren Sie ein responsives Produktraster:

- Desktop ($\geq 768\text{px}$): 3 Spalten nebeneinander, gleichmäßig verteilt
- Mobile ($< 768\text{px}$): 1 Spalte

Verwenden Sie ausschließlich Flexbox (kein Grid). Schreiben Sie das vollständige CSS für den Container `.product-grid` und die Karten `.product-card`.

Übung 3.B.2: Responsive Layout mit Flexbox — Lösung

```
.product-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}  
  
.product-card {  
  flex: 1 1 calc(33.33% - 1rem);  
  min-width: 0; /* verhindert Overflow bei langen Texten */  
  box-sizing: border-box;  
}  
  
@media (max-width: 767px) {  
  .product-card {  
    flex: 1 1 100%;  
  }  
}
```

`flex: 1 1 calc(33.33% - 1rem)` bedeutet: `grow=1`, `shrink=1`, Basisbreite $\approx$ ein Drittel minus halber Gap. `flex-wrap: wrap` erlaubt Zeilenumbruch. Auf Mobile wird die Basis auf 100% gesetzt $\rightarrow$ eine Karte pro Zeile.

Modul C — JavaScript

Übung 3.C.1: DOM-Manipulation und Events

Implementieren Sie folgende Funktion in JavaScript:

Ein Button `#toggle-theme` soll beim Klick die Klasse `dark-mode` auf dem `<body>` togglen. Zusätzlich soll der Button-Text zwischen „Dark Mode“ und „Light Mode“ wechseln.

Schreiben Sie den vollständigen JavaScript-Code (kein Framework).

Übung 3.C.1: DOM-Manipulation und Events — Lösung

```
const btn = document.getElementById('toggle-theme');

btn.addEventListener('click', () => {
  document.body.classList.toggle('dark-mode');
  const isDark = document.body.classList.contains('dark-mode');
  btn.textContent = isDark ? 'Light Mode' : 'Dark Mode';
});
```

Warum `classList.toggle`? Es fügt die Klasse hinzu, falls sie fehlt, und entfernt sie, falls sie vorhanden ist — in einer Operation, ohne manuelle Prüfung.

Übung 3.C.2: Fetch API und asynchrones JavaScript

Implementieren Sie eine Funktion `loadProduct(id)`, die:

1. Die URL `https://api.example.com/products/{id}` aufruft (GET)
2. Das JSON-Ergebnis in ein `<div id="product-detail">` rendert (Felder: `name`, `price`, `description`)
3. Fehler (Netzwerk oder HTTP-Status ≥ 400) mit einer Fehlermeldung im gleichen `<div>` anzeigt

Verwenden Sie `async/await`.

Übung 3.C.2: Fetch API und asynchrones JavaScript — Lösung

```
async function loadProduct(id) {
  const container = document.getElementById('product-detail');
  try {
    const response = await fetch(`https://api.example.com/products/${id}`);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
    const product = await response.json();
    container.innerHTML = `
      <h2>${product.name}</h2>
      <p><strong>Preis:</strong> €${product.price.toFixed(2)}</p>
      <p>${product.description}</p>
    `;
  } catch (error) {
    container.innerHTML = `<p class="error">Fehler: ${error.message}</p>`;
  }
}
```

Wichtig: `response.ok` prüft HTTP-Status 200–299. `fetch` wirft bei Netzwerkfehlern eine Exception, aber *nicht* bei HTTP 4xx/5xx — daher die manuelle `if (!response.ok)` Prüfung.

Modul D — Praktische JavaScript- Aufgaben

Übung 3.D.1: Luhn-Algorithmus — Kreditkarten-Validierung

Der **Luhn-Algorithmus** prüft, ob eine Kreditkartennummer formal korrekt ist (Tipp-fehler-Erkennung).
Algorithmus:

1. Letzte Ziffer = Prüfziffer, nicht verändern
2. Von rechts: jede zweite Ziffer verdoppeln
3. Wenn das Ergebnis ≥ 10 : beide Stellen addieren (z.B. $16 \rightarrow 1+6 = 7$)
4. Alle Ziffern aufsummieren
5. Wenn die Summe durch 10 teilbar ist $\rightarrow$ gültig

Implementieren Sie eine JavaScript-Funktion `isValidLuhn(number)`, die:

- Eine Kreditkartennummer als String entgegennimmt
- `true` zurückgibt wenn die Nummer den Luhn-Check besteht, sonst `false`

Testen Sie mit: 4532015112830366 (gültig) und 1234567890123456 (ungültig)



Übung 3.D.1: Luhn-Algorithmus — Kreditkarten-Validierung — Lösung

```
function isValidLuhn(number) {  
  const digits = number.split('').map(Number).reverse();  
  let sum = 0;  
  for (let i = 0; i < digits.length; i++) {  
    let d = digits[i];  
    if (i % 2 === 1) {           // jede zweite Stelle (von rechts, 0-indexiert)  
      d *= 2;  
      if (d >= 10) d = Math.floor(d / 10) + (d % 10);  
    }  
    sum += d;  
  }  
  return sum % 10 === 0;  
}  
  
console.log(isValidLuhn('4532015112830366')); // true  
console.log(isValidLuhn('1234567890123456')); // false
```

Erklärung:

- `reverse()` vereinfacht das "von rechts zählen": Index 0 = Prüfstelle (nicht verdoppeln), Index 1, 3, 5, ... werden verdoppelt.
- Verdopplung ≥ 10 : Quersumme = $\text{Math.floor}(d/10) + d\%10$ (äquivalent zu -9).
- Einsatz: Bankformulare validieren Kreditkartennummern **clientseitig** mit Luhn, bevor der Request gesendet wird — reduziert Serverlast und verbessert UX.

Übung 3.D.2: Web Workers und Local Storage

1. **Web Workers:** Ein Online-Shop berechnet Produktempfehlungen per komplexem Scoring-Algorithmus. Nach der Implementierung friert die UI kurz ein. Erklären Sie: a) Warum friert die UI ein? b) Welche Technologie löst dieses Problem, und wie funktioniert die Kommunikation zwischen UI und dem ausgelagerten Code?
2. **Lokale Datenspeicherung:** Ein Nutzer füllt ein mehrstufiges Registrierungsformular aus. Er schließt versehentlich den Tab. Nennen Sie zwei JavaScript-APIs für clientseitige Persistenz und vergleichen Sie:
 - Lebensdauer (bis wann gehen Daten verloren?)
 - Speicherkapazität (ungefähr)
 - Typischer Einsatz

Übung 3.D.2: Web Workers und Local Storage — Lösung

1. Web Workers

a) **Ursache:** JavaScript ist single-threaded. Lange synchrone Berechnungen blockieren den einzigen Thread — dadurch können keine UI-Events (Klicks, Scrolling) verarbeitet werden.

b) **Lösung: Web Worker** — führt JavaScript in einem separaten Thread aus. Kommunikation über `postMessage` (asynchron):

```
// main.js
const worker = new Worker('scoring.js');
worker.postMessage({ products: productList });
worker.onmessage = (e) => renderRecommendations(e.data);

// scoring.js (Web Worker)
self.onmessage = (e) => {
  const result = computeScoring(e.data.products); // rechenintensiv
  self.postMessage(result);
};
```

2. Clientseitige Persistenz

	sessionStorage	localStorage
Lebensdauer	Tab wird geschlossen → Daten weg	Bleibt erhalten (auch nach Browser-Neustart)
Kapazität	~5 MB	~5–10 MB
Typischer Einsatz	Temporärer Formularzustand pro Session	Nutzereinstellungen, Warenkorb-Entwurf



Für den Formular-Zwischenstand: localStorage — Daten überleben das Schließen des Tabs:

```
// Speichern
localStorage.setItem('formStep2', JSON.stringify(formData));
// Laden
const saved = JSON.parse(localStorage.getItem('formStep2'));
```